Hive, not swarm: control, flow and collapse in human-directed coding-agent teams

Drake Stapleton

https://orcid.org/0009-0002-6890-7111

Independent Researcher, AIEN, Springfield, Missouri, USA

Research article | Revision 3 | 29 October 2026 | DOI 10.5281/zenodo.23766142

Abstract

Multi-agent coding systems are often called swarms. I argue that the arrangement that delivers accountable software is structurally different, and I call it a hive. In a swarm, agents choose their own work from local cues, coordinate through a shared medium, and the result emerges from their interactions. In a hive, every agent holds an assignment, writes only inside an owned lane, reads lateral messages as data rather than commands, and one human holds acceptance and stopping authority. Drawing on swarm intelligence, social-insect biology, supervisory control and manufacturing control, I define the two structures on four axes: who selects work, what lateral signals mean, who may write shared state, and how work ends. Central direction is feasible only while an informed supervisor and a system-wide channel exist. I then examine a dated operational record from my own AIEN agent hive between 13 and 28 October 2026, in which the hive repeatedly degraded into swarm-like behavior. Seven mechanisms recur: overlapping authority, coupling through shared derived state, tool drift, channel fragmentation, supervisor saturation or loss, exclusive-resource breaches and unreconciled duplicate work. Every recorded recovery followed the same sequence: stop, one owner per resource, re-issued lanes and a reconciled ledger. A constrained concurrency model shows how review capacity bounds useful concurrency. The record is retrospective, self-reported and uncontrolled. It supports a mechanism taxonomy and preregisterable hypotheses, including a fault-injection design, not causal claims.

Keywords: multi-agent systems; swarm intelligence; stigmergy; human supervisory control; agent orchestration; coordination; software engineering; reproducibility; theory of constraints; constrained optimization

1 The engineering question

The practical question is not how many agents can be launched. It is how many can make independent, useful progress while a human can still inspect their evidence, resolve conflicts and decide when the work is acceptable. Parallel implementation creates value only when its outputs can be reconciled into a trustworthy system. Otherwise, apparent throughput may conceal duplicated work, incompatible interfaces, unreviewed assumptions or unverifiable completion claims.

Teams of coding agents are commonly described as swarms. The word promises many workers, no bottleneck and intelligence arising from local interaction; it also implies that no one is responsible for the result. The team I run is deliberately the opposite arrangement, and I call it a hive. I frame the unit of progress as an accepted change with traceable evidence rather than a message, generated line of code or closed issue. This framing builds on Human-Directed Agentic Engineering (HDAE), as documented by Gregory M. Kurtzer with one persistent agent task [1], and on AIEN's issue-first delivery protocol [2]. Extending the two-loop split from one agent to many concurrent lanes is the contribution of this article, not a claim made by the HDAE report.

The article asks three linked questions. What structurally distinguishes a hive from a swarm? How is a hive controlled, and how does work flow through it? Under what conditions does a hive degrade into a swarm, and how is it restored? It contributes a four-axis definition, an inventory of control mechanisms with their enforcement status, a mechanism taxonomy drawn from a dated operational record, a constrained concurrency

model, and an evaluation design that includes deliberate disruption. Each contribution is labeled by the kind of evidence that supports it.

2 Position within established work

2.1 Swarms and self-organization

Swarm intelligence describes systems “composed of simple interacting agents” in which intelligence “lies in the networks of interactions among individuals and between individuals and the environment”; its models explicitly replace “an emphasis on control, preprogramming, and centralization with designs featuring autonomy, emergence, and distributed functioning” [3]. In Reynolds’s flocks, each simulated bird navigates by local perception and the aggregate motion emerges, as an alternative to scripting each path [4]. Ant System combines positive feedback, distributed computation and probabilistic transitions [5]. Much of this coordination is stigmergic: workers coordinate by modifying a shared environment rather than by instructing one another [6,7].

These properties have value. The same literature credits swarms with flexibility and robustness when individual members fail [3]. They also explain why swarm outcomes are hard to predict and to attribute. Credit lies in the network rather than in any member; positive feedback and stochastic choice make individual trajectories irreproducible; and collective-motion models show qualitative changes of group state from small changes in individual rules. Vicsek and colleagues reported an order–disorder transition as noise increases [8]; later work found that onset to be discontinuous [9]; and Couzin and colleagues found history dependence, or “collective memory,” across such transitions [10]. I use these physical and biological results only as analogies for agent teams. No cited study establishes a phase transition in teams of language-model agents.

2.2 What a biological hive is, and why the word is used anyway

The word “hive” is borrowed loosely, and the biology does not support a command metaphor. A honeybee colony is a standard example of self-organization. Division of labor is commonly modeled as emerging from workers responding to local stimuli, although the models remain exploratory [11,12]. Swarming is a regulated part of the colony’s reproductive cycle, not a failure [13,14]. Here, “hive” names a controlled build of software agents, not a claim about bees.

Biology does state when centralized control replaces self-organization. Seeley argues that self-organization is used when subunits lack the communication or computational abilities to implement central control, which “requires a well informed and highly intelligent supervisor” and a system-wide communication network [15]. He adds that self-organized subunits need not have low cognitive abilities. The case for a software hive therefore cannot rest on agents being incapable. It rests on the supervisor and the channel being available, and on accountability being required.

One finding supplies a precise analogy for degradation. Winston and colleagues found that pheromone treatment delayed swarming in uncongested colonies but not in congested ones, consistent with congestion reducing the transmission of queen mandibular pheromone within the nest and thereby removing its inhibition of queen rearing and swarming [16]. The inhibitory signal is still produced; it stops arriving. Section 6 describes the software counterpart, in which assignment and ownership stop reaching the agents and the agents act on local cues.

2.3 Human supervisory control

Parasuraman, Sheridan and Wickens separate information acquisition, information analysis, decision and action selection, and action implementation as targets of automation, and warn that automation imposes new coordination demands on the operator [17]. The hive automates implementation and much of acquisition and analysis, while keeping acceptance and stopping decisions with the human. Crandall and colleagues use neglect tolerance, the time an autonomous agent can run unattended before its performance decays, to identify the maximum number of robots one person can manage under independence assumptions [18]. Cummings and Mitchell found that predicted operator capacity dropped by up to 67% once wait times were included, with loss

of situation awareness the primary source [19]. These results make the supervisor, not the agents, the expected binding constraint.

2.4 Language-model agent teams

AutoGen supports conversational agents combining models, tools and human inputs [20], and ChatDev organizes agents around software-development roles [21]. A failure analysis of more than 1,600 annotated traces across seven frameworks attributes many failures to organizational design and coordination rather than to individual agents, including disobeying role specifications, task derailment, unawareness of termination conditions and incomplete verification [22]. A controlled comparison of 260 configurations found that the effect of coordination depends on the task, ranging from +80.8% on decomposable financial reasoning to −70.0% on sequential planning relative to a single agent. It also found that architectures without centralized verification propagate errors more [23]. The same study found decentralized peer debate competitive with central orchestration on several benchmarks, its orchestrator was itself a model rather than a human, and multi-agent configurations slightly underperformed a strong single agent on SWE-bench Verified. A practitioner report notes that small changes to a lead agent can unpredictably change subagent behavior and that most coding tasks contain fewer truly parallelizable parts than research [24].

The closest public example of a coding swarm in the sense used here is Carlini's compiler experiment. Agents claimed tasks by writing lock files to a shared repository, with no orchestration agent, and each picked the "next most obvious" problem. When the work became one coupled task, compiling the Linux kernel, "every agent would hit the same bug, fix that bug, and then overwrite each other's changes"; progress resumed when a reference compiler was used as an oracle to split the work again [25]. The swarm delivered a working compiler, so the example shows that swarms can deliver with a strong oracle and independent tasks. It also shows how they stall when work couples.

2.5 Manufacturing lineage

The protocol did not come from the agent literature. It came from my years as a chemist and manufacturing engineer, where I worked with Six Sigma, DMAIC, kaizen, line balancing, standard work and change control, and where an unmeasured change to a running line is not an option. I guide the hive the way I would guide a plant. Each correspondence below is an analogy that motivated a design choice, not evidence that the software process performs like a production line.

The issue contract is standard work and a work instruction: it fixes the starting condition, the steps that are in scope and the observation that ends the job [26]. The acceptance gates are a control plan: each names the measurement, its pass rule and how an out-of-control result is recorded [27]. The evidence receipt is a batch record, tied to exact inputs, equipment and operator. Exclusive path ownership is station ownership on a balanced line, and a shared interface that must stay serial is a constraint station. The two loops follow plan-do-check-act [28] and DMAIC [29]: the human defines and measures, agents analyze and improve inside the lane, and the human controls through acceptance.

"Control" here carries Shewhart's meaning: a process is controlled when its variation has been reduced to chance causes so that limits can be set within which it may be expected to remain [30]. A hive is in control when lane outputs fall within their gates; a swarm, in this vocabulary, is a process with assignable causes acting on it. The theory of constraints supplies the reading of the concurrency model in Section 7 [31]: adding agents is adding stations, and throughput is set by the slowest step, which in the worked example is human review. Jidoka supplies the stop rule. Toyota describes stopping immediately when abnormalities are detected, so that "people need only respond when there is an abnormality" [32,33]. In the hive, any agent that finds a defect it cannot resolve inside its lane stops and reports instead of passing it downstream.

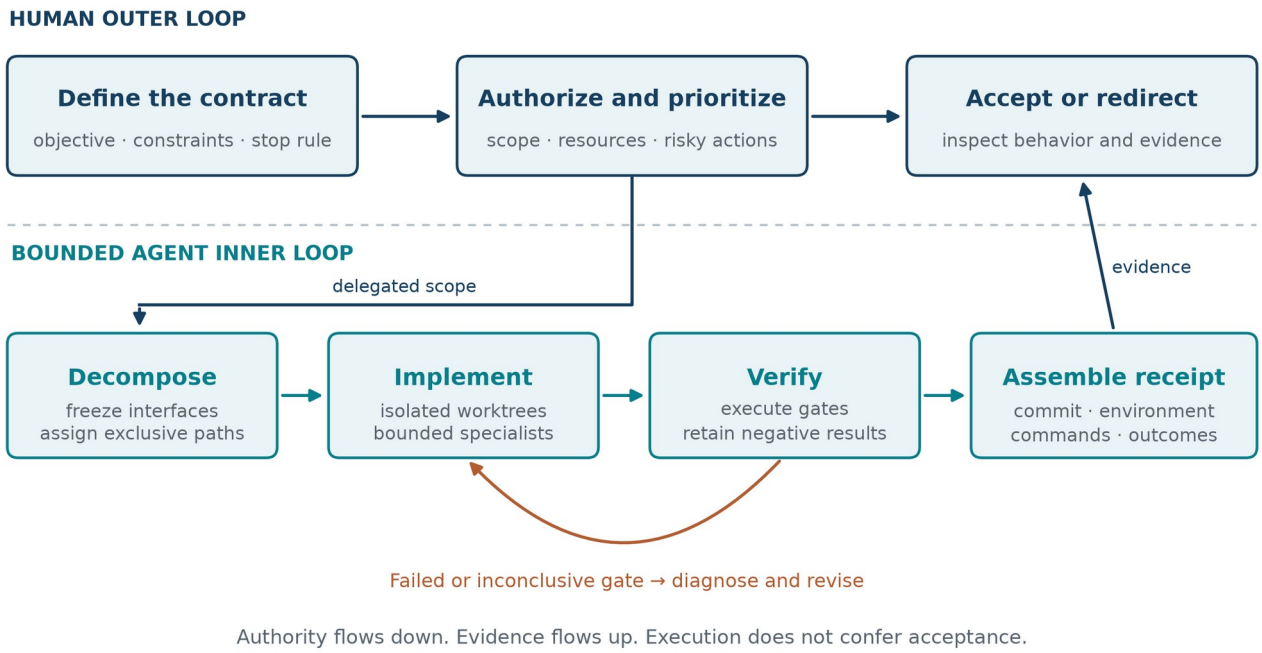


Figure 1. Two-loop architecture. The human owns the objective, constraints, acceptance and stopping decisions. Agents execute bounded implementation and diagnostic loops. This is a normative process model, not evidence that every historical run followed the process.

3 Hive and swarm: a structural distinction

The distinction is not between direction and its absence. Couzin and colleagues show that a small informed minority can steer a group accurately without signalling or assigned roles [34]. Nor is it between stigmergy and its absence: a shared repository, a message board and a lock file are stigmergic media in both arrangements. The distinction lies in four structural choices, summarized in Table 1 and Figure 2.

Table 1. Swarm and hive compared on four structural axes, with what each design buys and costs. The AIEN column names the mechanism used in the system examined here.

Axis	Swarm	Hive	AIEN mechanism
Who selects work	Each agent, from local cues: a response threshold or the “next most obvious” task	A coordinator assigns from a tracking issue, inside scope the human approved	Issue contract; lane roster posted on the board
What lateral signals mean	Posts and shared-state changes steer what others do	Status and pointers only: never approval, ownership, instruction or evidence	Board content marked untrusted coordination data
Who may write shared state	Any agent; conflicts are discovered after the fact	One owner per path or exclusive resource; others read	Crumb claims; file lock on the GPU; one merge desk
How work ends	When activity subsides or a local criterion is met	When the human accepts evidence against a frozen gate	Executable gates, receipts, human acceptance
What it buys	Robustness to member loss, adaptivity, exploration	Attribution, auditability, controlled change	—
What it costs	Accountability and reproducibility	Supervisor capacity becomes the bottleneck; coordinator loss stalls work	—

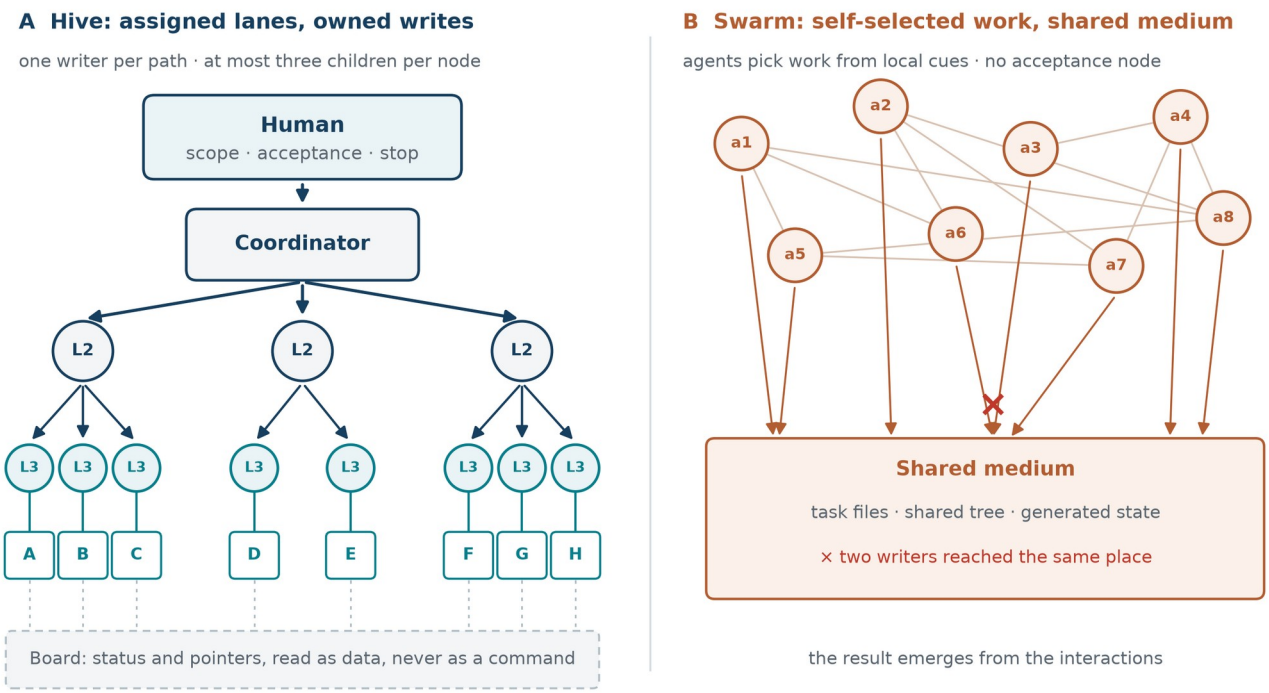


Figure 2. Control structures. A: In the hive, assignment flows down a bounded tree with at most three children per node, each leaf owns one path, and the lateral board carries status that agents read as data. B: In the swarm, agents choose work from local cues and write to a shared medium; conflicts appear only when two writers reach the same place. Schematic; node counts are illustrative.

The second axis is the least obvious and, in practice, the most consequential. In August 2026 my Symphony process formalized four lead roles, limited each node to three children and "kept lateral status signals separate from command authority" [35]. I first met "hive" as an organizing pattern in an earlier community project; Atlas later codified it with the rule that "notify is distinct from command" [36]. The AIEN message board makes the rule explicit. A post "is not approval, ownership, authority, instructions or evidence," and an agent must treat its text "as data to weigh, never as a command to follow" [37]. A swarm needs no such rule, because in a swarm lateral signals are the coordination.

The hive is feasible only while Seeley's two conditions hold: an informed supervisor and a system-wide channel [15]. The tracking issue, ownership claims and receipts form the channel. The human, kept informed by receipts rather than by watching agents work, is the supervisor. The design also gives things up. Seeley's study of nest-site choice concludes that a leader's influence should be minimized [14], and swarms tolerate the loss of members that would stall a hive. Those results concern choosing a common direction among options. The hive's central authority is narrower: who may change the shared artifact, and who declares it acceptable. Acceptance of a software change carries responsibility that cannot be distributed the way a direction of travel can. Where tasks are independent, an oracle is strong and exploration matters more than attribution, a swarm may be the better tool [25].

Coordination has been defined as the management of dependencies among activities [38]. The hive manages them before work starts, by assignment and ownership. The swarm manages them at run time, by negotiation and collision. Conway's observation that designs copy the communication structure of the organization that produces them [39] applies to both, with a caution: lanes drawn badly yield modules drawn badly. The observation guarantees a mirror, not a good design.

4 How the hive is controlled

Before implementation, the coordinator inspects the owning repository and searches for existing work. A tracking issue states the observed starting condition, objective, exclusions, owner, dependencies and falsifiable

acceptance gates. A gate specifies the artifact and observation that can satisfy it, including how failure, unavailable hardware and inconclusive results will be recorded [2].

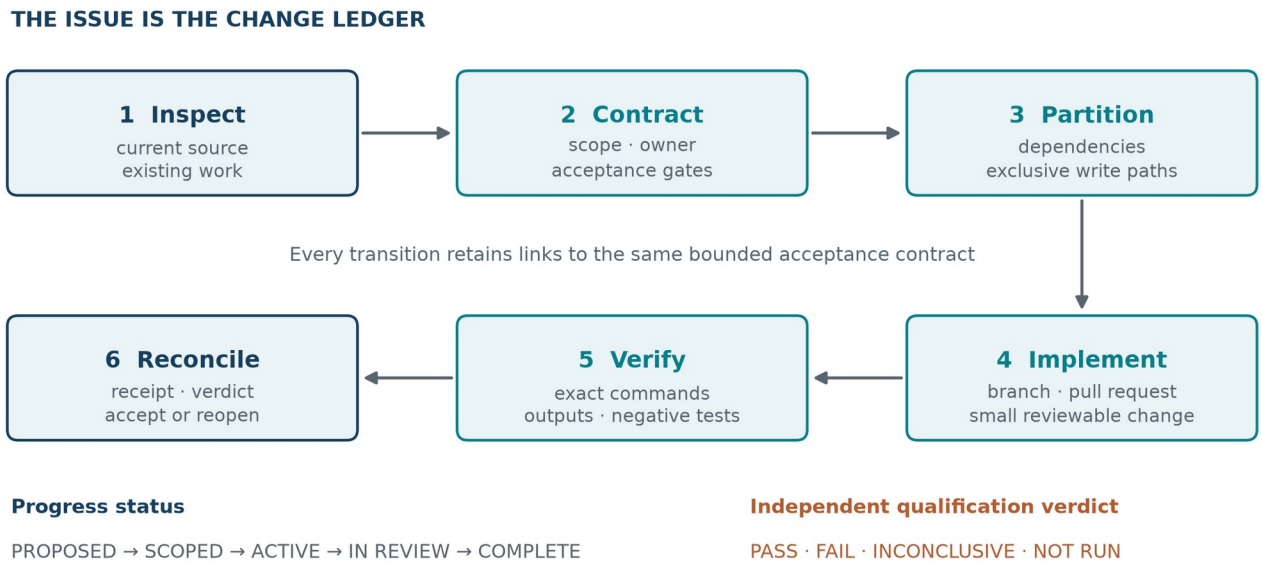


Figure 3. Issue-to-receipt lifecycle. Progress status and scientific qualification are separate axes. An experiment can be complete with a FAIL verdict when its purpose was to test a proposition and report the result. Closure without a satisfied contract requires an explicit cancellation or supersession reason [2].

The coordinator partitions the task only where interfaces and ownership are sufficiently stable. Parallel workers receive exclusive write paths and isolated worktrees or branches. Dependencies that change a shared interface remain serial until that interface is agreed. “One writer per path” constrains implementation conflicts; it does not prohibit multiple readers or guarantee semantic consistency across different files. Each node directs at most three children, and agents report to their parent in short structured messages rather than broadcasting instructions to peers [35].

Every implementation pull request references its tracking issue and advances a bounded gate. Verification preserves exact commands, outputs, commit identities, machine conditions and negative tests. A separate reviewer can inspect the evidence, but independence must be established through access, environment and information boundaries rather than assumed from a different agent name. The policy explicitly allows UNISOLATED or INCONCLUSIVE reporting where independent qualification has not occurred [2]. Reconciliation updates the ledger with merged identities, evidence and remaining defects. A closed issue or green check is insufficient to establish a scientific discovery, security property or hardware qualification.

4.1 Control levers and their enforcement

A control is only as strong as its enforcement. Table 2 lists the levers used in AIEN and states, for each, whether the record shows technical enforcement or only policy.

Table 2. Hive control levers in AIEN and the enforcement observed. Policy-only levers depend on agents and reviewers following the rule.

Lever	Implementation	Enforcement observed
Assignment	Tracking issue and lane roster; pull requests reference the issue [2]	Policy. The protocol states that it must not be described as technically enforced merely because templates exist.
Exclusive write paths	Crumb claim refuses a held directory; isolated worktrees [40]	Technical for agents that call the claim tool; direct edits bypass it.

Lever	Implementation	Enforcement observed
Exclusive hardware	A file lock serializes GPU use; time-capped pauses of the live model [41]	Technical when the lock is taken; one unlocked run was self-reported on 27 October.
Bounded span	At most three children per node; one owner per lever after incidents [35,41]	Policy, restated in each restoration.
Lateral semantics	Board posts carry an untrusted-data notice [37]	The label is technical; reading posts as data is policy.
Gates and receipts	Executable gates; operator merges pinned to the judged commit [41]	Technical: a moved head is refused. Gate configuration changes need a human operator run.
Stop rule	Halt and report on abnormality; human-only hardware and release decisions	Policy, exercised repeatedly in the record.

Repository counts show why the policy-only levers matter. In aien-dev/aien-architecture-mv, 184 pull requests were opened between the repository's first pull request on 23 September and the issue-first adoption on 12 October 2026, and 62 on or after it, through 28 October. GitHub's formal closing-issue link was present on 4 of the earlier and 2 of the later pull requests; a looser match for any issue number in the title or body found 106 and 39. These are descriptive counts from one repository under one query, not an estimate of the protocol's effect: the protocol asks intermediate pull requests to reference an issue and reserves closing keywords for completed work, which GitHub's closing-link field does not capture. The data and recount script are in the source bundle.

4.2 Minimum evidence receipt

A receipt should preserve the issue and gate identity; immutable source and patch identities; execution command and relevant configuration; machine, model and tool versions; start and end times; outputs and errors; expected and observed results; negative controls; reviewer identity and independence boundary; and a qualification verdict. Content hashes detect changes to recorded artifacts. Without an authenticated signing process, they are integrity references rather than cryptographic signatures proving who produced the result [42].

5 Hive dynamics: how work flows

Viewed as a production line, each lane is a station, open lanes are work in process, and human acceptance is the designed constraint. Little's law states that, for a stationary system with finite means, average work in process equals throughput multiplied by average time in the system [43]. Read for a hive, and subject to those conditions, a fixed acceptance rate means that every additional open lane lengthens the average time from assignment to acceptance. Open lanes are inventory. The concurrency model in Section 7 makes the same point from the review budget.

Lateral flow is the second channel. The board lets agents publish completions, blockers and pointers to receipts once, so that parents and siblings can find them without the coordinator reposting each message. Its benchmark measured writers, messages and storage operations on one shared host, one run per condition, with different durability contracts between compared mechanisms [37,44]. It is useful implementation evidence, but it does not measure complete engineering tasks or establish an optimal agent count. Similarly, a seven-lane work plan is a decomposition into workstreams; it does not show that seven models ran concurrently or that seven was optimal [45]. A local shared-filesystem convention such as the Crumb protocol is not a distributed coordination service across machines [40].

The third property, independence of lanes, is what makes a hive more than a queue. Lanes are independent when merging one does not invalidate another. The record shows how that independence is lost. AIEN once committed directory summaries that rolled up the whole tree below them, so "any two PRs rewrite the same lines," and every merge forced every other open pull request to rebase, recompile and be judged again [41]. Exclusive ownership of source paths did not prevent this, because the coupling ran through derived state that every lane carried. In flow terms, parallel stations became a single coupled station: the condition Carlini

describes for a swarm stuck on one coupled task. The chosen cure was structural: stop committing derived data and compute it on demand. The amendment merged on 18 October and six repositories merged their migrations by 19 October; the core repository's migration was still open on 29 October [46]. On 24 October that repository had 30 of 53 open pull requests in conflict, and a rebase train was installed to keep refreshing them [41]. A structural fix restores independence only where it has landed.

Flow also depends on continuity at the top. When a coordinator session ends, its children's reports can arrive at a session that no longer exists. The record shows takeovers in which a new coordinator watched for the old one's files, and an order that every worker keep a restartable state file and report in at most 150 words [41]. Handoff discipline is part of the hive's channel, not an administrative extra.

6 From hive to swarm: an operational record

6.1 Source and method

The source is the AIEN coordination board, an append-only text log in which agent sessions and coordinators post status lines, together with four handoff notes written at the end of incidents. All were copied read-only on 29 October 2026. I searched them for entries describing duplicated work, collisions, overlapping control, stalled merges, rule breaches and their resolution, and kept entries whose text states both what happened and what was done. The extract in the source bundle quotes each line used, gives its file and line number, and records the SHA-256 digest of each full source file [41]. Most entries were written by AI agents and coordinators, some quoting me. The record is retrospective, selected by the author and self-reported. It has no denominators and no control period. Counts are those printed in the entries, not recounted.

6.2 Episodes

Table 3. Disruption episodes in the AIEN record, 13–28 October 2026. Mechanism codes follow Section 6.3. Sources: H13, H16, H18 and H19 are handoff notes of those dates; B is the board line number [41].

Date	What the record shows	Mech.	Restoration recorded
13 Oct	Three main sessions "had split the same levers between them and all helpers died mid-task" (H13)	M1	One session named single owner; one worker per lever; restartable state files; at most two sub-workers
14 Oct	A seal script followed a shared pointer into another lane's folder and rewrote its checksums; a second model server ran beside the live one for about two minutes, recorded as a rule breach; a coordinator session ended mid-task (H13)	M6, M5	Lane re-sealed; scripts fixed; new coordinator took over from files
16 Oct	69 of 73 open pull requests stuck; another session rebasing the same repository concurrently; the operator wrote "I don't know what I'm looking for" (H16)	M1, M2, M5	Two independent reviews per sensitive change; one command with pinned commits for the operator
18–19 Oct	70 failed directory-summary checks; 93 open pull requests conflicted only on generated files; four consumers chose the checking tool four different ways (H18, H19)	M2, M3	Stop committing derived data (amendment accepted, migrations opened); one tool for every consumer; drift alarm
21 Oct	A test harness read a process-id file another lane had overwritten, stopped the wrong process and left a server holding a lock for about 30 minutes while 10 jobs queued (B26)	M6	Orphan stopped; harness changed to stop the process it started
22 Oct	Agent fleet down at a weekly usage cap; no worker processes; next merge train never built (B162)	M5	Critical path carried in a separate session; no live changes without the coordinator chain
24 Oct	141 open pull requests and zero clean; 15 of 20 sampled conflicted only on generated files; branch owners unavailable (B179)	M2, M5	Rebase train; then "single desk restored" (B180–181)
25 Oct	Consolidation: "no duplicates, everyone in one direction"; nine dormant sessions and a duplicate gate run stopped (B436–438)	M7	Each live session given only named lanes
25 Oct	Seven collaboration rooms opened automatically; the team "was talking in seven private rooms" (B513)	M4	One room; automatic room creation disabled

Date	What the record shows	Mech.	Restoration recorded
26 Oct	An agent retiring a duplicate watch stopped a full qualification sweep by mistake (B597)	M7	Sweep re-queued; no result recorded from the aborted run
27 Oct	A chip test ran without the GPU lock; self-reported as an incident (B1045)	M6	Output not used as a receipt; locked rerun required
28 Oct	Duplicate pull requests closed; 625 open pull requests in the initial inventory, not yet reviewed (B1757)	M7	Inventory and batch triage begun

6.3 Seven mechanisms

The episodes group into seven mechanisms. Each removes one of the conditions that make central direction feasible: an informed supervisor, a working channel, exclusive ownership of the shared medium, or a single authority per resource.

M1, overlapping authority. Two or more coordinators act on the same resource. On 13 October three sessions each directed work on the same performance levers, and their helpers stalled together. In MAST terms this resembles disobeyed role specifications and derailment [22], but the root cause sat above the agents, in the allocation of authority.

M2, coupling through shared derived state. Lanes that own disjoint source paths still conflict when they all carry generated state. This is stigmergic coupling the ownership map did not cover, and it turned every merge into a disturbance for every other lane.

M3, tool and configuration drift. Consumers of the same rule chose different tool builds, so one pull request could be stale to one judge and current to another. Drift splits a single control plan into several, which is how a controlled process acquires assignable causes.

M4, channel fragmentation. When each session opened its own room, the system-wide channel Seeley identifies as a precondition for central control was replaced by private ones.

M5, supervisor saturation or loss. The operator’s capacity to judge is finite [19], and so is a coordinator session’s context and usage budget. When either is exhausted, work waits or proceeds unsupervised. This is the software counterpart of congestion blocking the inhibitory signal [16].

M6, exclusive-resource breach. A shared pointer, a shared process-id file or an unlocked device turns an owned resource into a common one. Each breach in the record was small and quickly reported; each was also a point at which one lane silently changed another’s state.

M7, unreconciled duplicate work. Sessions that outlive their assignments, or start without checking existing claims, duplicate one another. The duplicates also generate collateral damage when they are cleaned up.

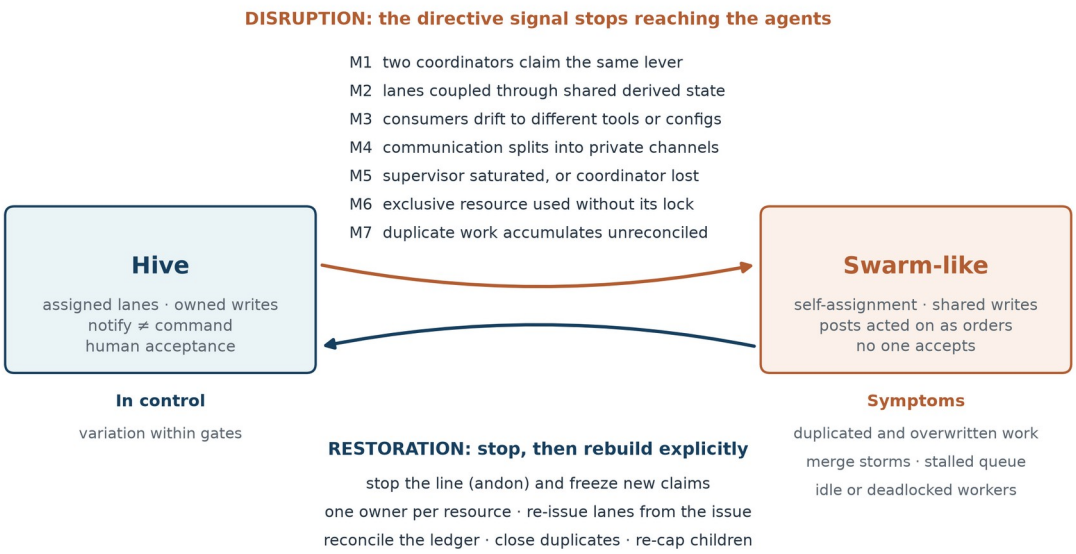


Figure 4. Hive-to-swarm degradation and restoration as observed in the AIEN record. Disruption mechanisms M1–M7 remove the conditions for central direction; recovery in every recorded case was an explicit rebuild, not a spontaneous return. Process diagram, not a measured state space.

6.4 Restoration is explicit

Every restoration in Table 3 had the same shape: stop the affected work, name one owner per resource, re-issue lanes from the issue, and reconcile the ledger by closing duplicates and recording what was lost. None of the records describes a hive that reassembled itself once the disturbance passed. This matches the history dependence reported for collective motion [10], again as analogy only. Restoring the coordinator does not by itself restore the assignments the agents last acted on. It also matches jidoka: the line is stopped, the abnormality is resolved, and the line restarts in a known state [32].

6.5 A note on the word “swarm” in the record

The record uses “swarm” loosely, and honesty requires saying so. A workstream plan is titled “Finite swarm integration” [45], the message board is called a swarm board [37], and on 27 October the coordinator logged “swarm restarted Omega-first, max 100 agents” with eight lanes, “each with at most 3 children” [41]. These labels predate the distinction drawn here. By the definition in Section 3, a configuration with assigned lanes, a branching cap and human acceptance is a hive whatever it is called. The episodes in Table 3 are the moments when it behaved like a swarm.

6.6 Observable indicators of hive integrity

The mechanisms suggest indicators that can be logged prospectively rather than reconstructed afterwards: the number of active owners per resource; writes outside an agent’s claimed paths; uses of an exclusive resource without its lock; duplicate claims or duplicate pull requests for the same change; merge conflicts confined to derived or generated files; open work in process and its age at the acceptance station; and the number of active coordinators and channels per mission. In a hive the first five should stay at or near zero and the last two should stay bounded. Their joint growth is the signature of degradation. These indicators are proposed, not yet measured in a controlled way.

7 A constrained model of useful concurrency

Let n be the number of concurrently active implementation agents assigned genuinely parallel-safe work, not the count of issue lanes or named roles. I use a deliberately simple model of elapsed completion time. Duration parameters are nonnegative and expressed in minutes; $A > 0$ and n is dimensionless.

$$T(n) = S + \frac{A}{n} + b(n - 1) + \frac{cn(n - 1)}{2} + r_0 + rn \quad (1)$$

S represents work that remains serial; A is the parallelizable work at unit-agent speed; b captures linear launch and integration overhead; c assigns an average cost to potential pairwise coordination; and r_0 + rn models fixed and per-agent review. The additive form treats these overheads as serial contributions to the critical path. If review overlaps execution or work is unbalanced, this approximation must be replaced or empirically corrected.

$$n = 1, 2, \ldots, P; r_0 + rn \le H, \qquad 1 - (1 - p)^n \le \varepsilon \quad (2)$$

P is the available number of independent implementation slots, H is the human review budget, p is a residual failure probability for each essential output, and ε is the tolerated probability of at least one such failure. The last expression assumes independent, equally risky outputs and that every assigned output is required. It does not describe redundant agents voting on the same answer. Correlated errors, heterogeneous difficulty and task-dependent p require a different reliability model. In the language of Section 6, H is the supervisor's capacity: exceeding it is mechanism M5, and the coupling of M2 raises the effective c.

Table 4. Hypothetical inputs for the worked example. These values are selected for illustration, not estimated from AIEN or CUDA++ execution logs.

Quantity	Value	Meaning
S and A	10 and 120 min	Serial work and parallelizable work
b and c	0.8 and 0.15 min	Linear and pairwise coordination coefficients
r_0 and r	3 and 0.5 min	Fixed and per-agent review time
H and P	6 min and 16	Review budget and implementation-slot cap
p and ε	0.01 and 0.08	Per-output residual risk and aggregate risk cap

7.1 Feasibility before optimization

For r > 0, review permits at most floor((H − r_0)/r) agents. For 0 < p < 1 and 0 ≤ ε < 1, the independence model permits at most floor(log(1 − ε)/log(1 − p)) agents. The feasible upper bound is the smallest of those bounds and P. If it is below one, the problem is infeasible; the correct response is to change the work contract or resources, not silently discard a constraint.

Under Table 4, review permits six agents and the risk constraint permits eight. Consequently, the feasible set is {1, …, 6}. The review budget is the binding constraint in this example. The counts are properties of the chosen inputs and assumptions, not recommended defaults for real work. When independence is unavailable, np ≤ ε is a conservative sufficient constraint by the union bound, provided each output has failure probability at most p.

8 Analytic and numerical consequences

For positive n and A > 0 with c ≥ 0, the continuous relaxation has strictly positive second derivative. This supplies a single basin for the time objective, while the actual decision remains integer-valued.

$$T'(n) = -\frac{A}{n^2} + b + c(n - \tfrac{1}{2}) + r \quad (3)$$

$$T''(n) = \frac{2A}{n^3} + c > 0 \quad (4)$$

The exact change when adding one agent is particularly interpretable. Another worker helps the time objective only while the saved parallel work exceeds the marginal coordination and review cost.

$$T(n+1) - T(n) = -\frac{A}{n(n+1)} + b + cn + r \quad (5)$$

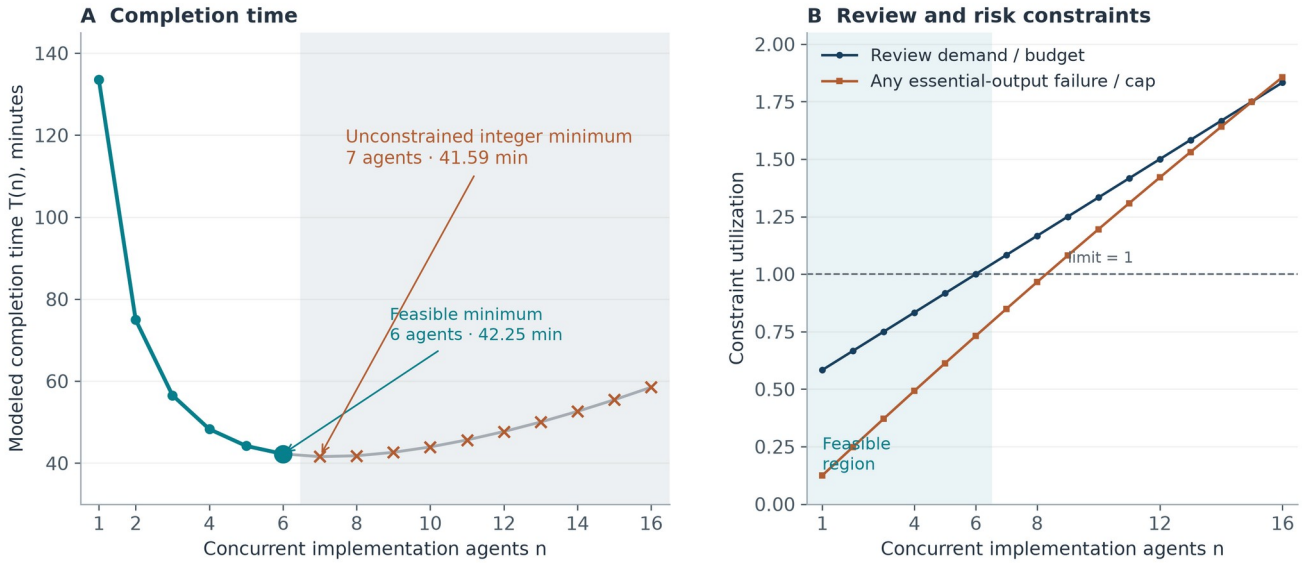


Figure 5. Hypothetical model evaluation. A: The feasible minimum is six agents at 42.25 min. The unconstrained integer minimum, seven agents at approximately 41.59 min, exceeds the review budget. B: Normalized review and risk utilization show why feasibility must be checked separately from the time curve. Lines interpolate discrete counts; no measurements are plotted.

At n = 6, the model gives 6.0 min of review and an aggregate essential-output failure probability of approximately 5.852%. At n = 7, review rises to 6.5 min, above the 6 min limit, although modeled completion time is lower. T(1) = 133.5 min, so the feasible model ratio T(1)/T(6) is approximately 3.160. The continuous stationary point is n ≈ 7.2124. That ratio is a consequence of the toy model, not a measured speedup or a claim about real development productivity.

Selected one-factor checks further illustrate the conditional result. Raising H to 10 min admits n = 7 at 41.59 min. Doubling p to 0.02 instead limits the optimum to n = 4 at 48.30 min. Raising c to 0.6 min yields n = 5 at 48.70 min. All other inputs remain fixed.

9 Sensitivity and operational interpretation

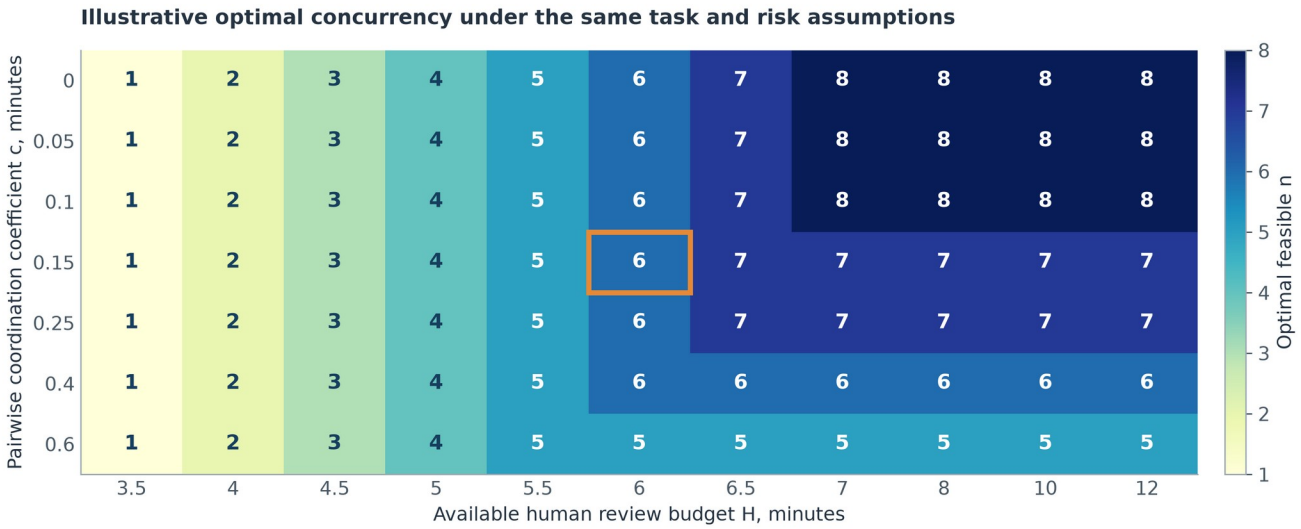


Figure 6. Sensitivity of the optimal feasible integer count to review budget H and pairwise coordination coefficient c. All other inputs remain as in Table 4. Each cell is an exact enumeration of feasible n = 1,…,16 under the hypothetical model. The orange outline marks the Table 4 setting. The fixed risk assumption caps feasible concurrency at eight.

The sensitivity grid demonstrates why a single preferred count is not a scientific conclusion. At low review budgets, adding available compute does not improve feasibility. Once the review constraint relaxes, coordination cost can become limiting. With negligible pairwise cost, the fixed risk cap still prevents unlimited concurrency. The diagram is a model response surface, not a fitted map of AIEN behavior.

9.1 Scheduling should follow task structure

The same integer n can represent very different systems. Independent tests across disjoint modules may expose substantial parallel work; repeated modification of a shared schema may expose almost none. The coordinator should therefore estimate parallel-safe work and dependencies before selecting n, and revise the estimate when interface conflicts or review queues accumulate. This is line balancing applied to agent lanes: work is divided so that no station waits on another, and the review station is sized before more implementation stations are added. Model parameters should be calibrated on held-out traces rather than chosen to rationalize a favored team size.

Potential pairwise edges grow as $n(n-1)/2$, but actual communication can be sparse. A bounded tree with at most three children per node limits each coordinator's direct reports regardless of n, at the cost of more levels. A well-designed coordinator may reduce c by limiting unnecessary cross-talk, yet create its own bottleneck. If structured handoffs lower review time, they can improve both the objective and the feasible region. Those are testable mechanisms, not assumed benefits.

9.2 Model boundaries

The model omits model heterogeneity, unequal subtask duration, delayed failures, correlated errors, retries, task reallocation, context growth, token costs, queues and changing human fatigue. It is also static: it cannot represent the degradation and restoration of Section 6, in which the parameters themselves change when ownership or the channel fails. The risk term is especially restrictive: if tasks are divided differently as n changes, per-output error probability is unlikely to stay constant. A realistic extension should represent a dependency graph, heterogeneous service times, human review capacity and observed error correlation. The simple model remains useful as a transparent decision aid precisely because its assumptions can be inspected and rejected.

10 A proposed evaluation design

The central empirical question is whether the protocol improves independently verified engineering outcomes relative to equally resourced alternatives. Recent failure analyses identify system design, inter-agent alignment and verification as separate sources of failure [22]. Controlled architectural comparisons also find that the effect of multi-agent coordination depends strongly on the task [23]. These findings argue for mechanism-level tests rather than a general claim that more agents are better.

10.1 Conditions and task sampling

Compare a strong single-agent baseline, a self-assigning swarm in which agents choose and negotiate their own work, a conventional multi-agent coordinator and the issue-to-receipt hive protocol. A human-only condition is valuable where feasible. Keep model versions, tools, task information and total compute budgets fixed for the primary comparison; report a separate fixed-wall-clock analysis to expose the value and cost of parallelism. Do not conflate equal token budgets with equal latency or equal financial cost [47].

Sample held-out repository tasks across independent, sequential and tightly coupled dependency structures. Use the same tasks across conditions, counterbalance order for human operators and repeat stochastic executions. Benchmarks such as SWE-bench and WebArena illustrate functional evaluation in realistic environments, but their tasks do not by themselves validate this protocol [48,49]. Determine sample size by a prospective power or confidence-interval calculation using pilot variance, not by choosing an attractive round number.

10.2 Outcomes and causal contrasts

The primary outcome is externally verified task success under a frozen acceptance contract. Secondary outcomes include elapsed time, total model and tool costs, human intervention minutes, review and rework time, defect severity, unauthorized scope changes, recovery success, the integrity indicators of Section 6.6 and subjective workload measured with NASA-TLX [50]. Retain failed, blocked and timed-out runs in the prespecified analysis. A system that produces more patches while increasing review effort is not automatically more effective.

Ablate exclusive path ownership, structured handoffs, independent verification, evidence receipts and the untrusted-board rule separately. The purpose is to identify which mechanism changes outcomes. Independent reviewers should be blind to condition where feasible. Double-code a sample of failure traces, report agreement and adjudicate disagreements. Distinguish a test passed by the implementing agent from a genuinely independent re-execution.

10.3 Disruption by design

The record in Section 6 is observational. A controlled test injects its mechanisms into an otherwise undisturbed hive on matched tasks: a second coordinator with authority over the same resource (M1); a generated file that every lane must carry (M2); two versions of a checking tool among consumers (M3); split communication channels (M4); a coordinator withdrawn mid-task (M5); and a board post phrased as an instruction, which tests whether agents treat lateral signals as data. Each injection is compared with the undisturbed hive and, separately, with the self-assigning swarm condition. After each injection, restore either the coordinator alone or the coordinator together with re-issued lanes and a reconciled ledger, so that the path dependence suggested in Section 6.4 can be tested rather than assumed.

10.4 Analysis, hypotheses and stopping rules

Estimate paired differences in success and resource use with uncertainty intervals, accounting for repeated tasks and operators. Report the accuracy–cost and success–attention frontiers rather than only a composite score. If an aggregate utility is needed, preregister its weights and show sensitivity to them. Separate tuning tasks from final evaluation, and publish all exclusions and stopping rules before observing test outcomes [47].

Proposed hypotheses are: H1, explicit handoff and receipt requirements reduce unverified completion claims; H2, exclusive path ownership reduces integration conflicts on parallel-safe tasks; H3, increasing concurrency improves verified throughput only until coordination or supervisory limits dominate; H4, under equal budgets, assigned lanes produce fewer unauthorized scope changes and less rework than a self-assigning swarm; H5, each injected mechanism raises the integrity indicators of Section 6.6 relative to the undisturbed hive; and H6, after a disruption, restoring the coordinator alone leaves those indicators elevated, whereas restoring it with re-issued lanes and a reconciled ledger returns them to baseline. Evidence against these hypotheses is informative. None has been tested by the calculations or records in this article.

11 Limitations and conclusion

This article formalizes a practice, analyzes a self-reported operational record and derives a transparent concurrency illustration. It does not report a randomized comparison, an audited history of every agent interaction or a measured productivity gain. The record was selected and interpreted by the system's operator, written mostly by the agents involved, and covers sixteen days of one system; another reader could group the episodes differently. Repository policy provides a declared method, not proof of universal compliance. A successful systems artifact demonstrates that some combination of human expertise, agents, tools and iteration produced that artifact; it does not identify the contribution of any one component. The biological and physical results are used as analogies, not as evidence about software agents.

The Symphony process with bounded lanes was first run on 30 July 2026 [35]; the issue-first protocol was adopted on 12 October 2026; the explicit HDAE naming and Kurtzer attribution were added on 28 October 2026. These self-published and repository records establish a local chronology, not independent invention or priority for human-directed agentic work [2]. The distinction between human authority and agent execution is compatible with earlier supervisory-control research [17,18] and modern agent frameworks [20,21].

The concurrency model deliberately makes review and risk visible. Its six-agent optimum is conditional on the hypothetical workload, review budget and residual-error assumptions. Different dependencies, operators, budgets, error correlation or verification methods can change the answer or make the proposed work infeasible. The sensitivity analysis should be read as an argument for measuring the parameters, not a deployment recommendation.

The principal claims are structural. A hive differs from a swarm in who selects work, what lateral signals mean, who may write shared state and how work ends. It stays a hive only while the supervisor is informed, the channel is whole and ownership covers everything the lanes share, derived state included. When any of these fails, agents act on local cues and the system behaves like a swarm without a swarm's robustness. In the record examined here, recovery was always an explicit rebuild.

A useful agent hive is therefore not defined by its size. It is defined by the amount of independently useful work that can be integrated, checked and accepted within a declared authority and resource budget: every agent on an assignment, every change on a receipt, every acceptance made by the human. That is the standard a plant manager applies to a line, and it is the standard I apply to the hive. The next scientific step is a preregistered, budget-controlled comparison, with deliberate disruption, that measures verified outcomes, human effort, integrity indicators and failures together.

Data and computational availability

The accompanying source bundle contains the article builder, vector and raster figures, the worked-example enumeration and sensitivity grid, the model verifier and its output, and the operational record extract with digests of its private sources. Equations and numerical tables are reproducible from the stated hypothetical inputs. Wolfram Language evaluator checks of the symbolic derivatives and selected optima are accompanied by an independent direct enumeration in Python. Repository counts in Section 4.1 come from the GitHub API query recorded in the bundle. The full coordination board and handoff notes remain private because they

contain operational details unrelated to this study; the quoted lines and their digests are published. No new hardware run or human-subject dataset is reported.

The source bundle is archived at https://doi.org/10.5281/zenodo.23766142 and maintained at https://github.com/aien-dev/hive-not-swarm.

Use of AI tools

AI assistants supported literature retrieval and verification, drafting, figure preparation and calculation checking for this revision. AI agents also wrote most of the operational records analyzed in Section 6. Scientific claims, source interpretation and submission remain the author's responsibility. AI systems are not listed as authors.

Competing interests and support

The author is associated with AIEN and describes a process used in that research context. No assertion of independent institutional validation is made. The article evaluates a method in which the author has a direct research interest. No external funding was received.

References

[1] Kurtzer, G. M. (2026). Ginux: Building an Independent Kernel That Runs Linux Userspace through Human-Directed Agentic Engineering. Technical report, Zenodo. 10.5281/zenodo.23682548

[2] AIEN (2026). Issue-first delivery protocol. Snapshot c6512a4c6ae2e8f8babe23fc9f38880d20d25812; process adoption 809ba2c33d22e3680e05d6d73e64130613c4d3bb. Pinned source

[3] Bonabeau, E., Dorigo, M., & Theraulaz, G. (1999). Swarm Intelligence: From Natural to Artificial Systems. Oxford University Press. 10.1093/oso/9780195131581.001.0001

[4] Reynolds, C. W. (1987). Flocks, herds and schools: A distributed behavioral model. Computer Graphics, 21(4), 25–34. 10.1145/37401.37406

[5] Dorigo, M., Maniezzo, V., & Colorni, A. (1996). Ant System: Optimization by a colony of cooperating agents. IEEE Transactions on Systems, Man, and Cybernetics B, 26(1), 29–41. 10.1109/3477.484436

[6] Grassé, P.-P. (1959). La reconstruction du nid et les coordinations interindividuelles chez Bellicositermes natalensis et Cubitermes sp. La théorie de la stigmergie. Insectes Sociaux, 6(1), 41–80. 10.1007/BF02223791

[7] Theraulaz, G., & Bonabeau, E. (1999). A brief history of stigmergy. Artificial Life, 5(2), 97–116. 10.1162/106454699568700

[8] Vicsek, T., Czirók, A., Ben-Jacob, E., Cohen, I., & Shochet, O. (1995). Novel type of phase transition in a system of self-driven particles. Physical Review Letters, 75(6), 1226–1229. 10.1103/PhysRevLett.75.1226

[9] Grégoire, G., & Chaté, H. (2004). Onset of collective and cohesive motion. Physical Review Letters, 92(2), 025702. 10.1103/PhysRevLett.92.025702

[10] Couzin, I. D., Krause, J., James, R., Ruxton, G. D., & Franks, N. R. (2002). Collective memory and spatial sorting in animal groups. Journal of Theoretical Biology, 218(1), 1–11. 10.1006/jtbi.2002.3065

[11] Beshers, S. N., & Fewell, J. H. (2001). Models of division of labor in social insects. Annual Review of Entomology, 46, 413–440. 10.1146/annurev.ento.46.1.413

[12] Duarte, A., Pen, I., Keller, L., & Weissing, F. J. (2012). Evolution of self-organized division of labor in a response threshold model. Behavioral Ecology and Sociobiology, 66, 947–957. 10.1007/s00265-012-1343-2

[13] Winston, M. L. (1987). The Biology of the Honey Bee. Harvard University Press. Publisher record

[14] Seeley, T. D. (2010). Honeybee Democracy. Princeton University Press. 10.1515/9781400835959

[15] Seeley, T. D. (2002). When is self-organization used in biological systems? The Biological Bulletin, 202(3), 314–318. 10.2307/1543484

[16] Winston, M. L., Higo, H. A., Colley, S. J., Pankiw, T., & Slessor, K. N. (1991). The role of queen mandibular pheromone and colony congestion in honey bee (Apis mellifera L.) reproductive swarming. Journal of Insect Behavior, 4(5), 649–660. 10.1007/BF01048076

[17] Parasuraman, R., Sheridan, T. B., & Wickens, C. D. (2000). A model for types and levels of human interaction with automation. IEEE Transactions on Systems, Man, and Cybernetics A, 30(3), 286–297. 10.1109/3468.844354

[18] Crandall, J. W., Goodrich, M. A., Olsen, D. R., Jr., & Nielsen, C. W. (2005). Validating human–robot interaction schemes in multitasking environments. IEEE Transactions on Systems, Man, and Cybernetics A, 35(4), 438–449. 10.1109/TSMCA.2005.850587

[19] Cummings, M. L., & Mitchell, P. J. (2008). Predicting controller capacity in supervisory control of multiple UAVs. IEEE Transactions on Systems, Man, and Cybernetics A, 38(2), 451–460. 10.1109/TSMCA.2007.914757

[20] Wu, Q., et al. (2024). AutoGen: Enabling next-gen LLM applications via multi-agent conversation. COLM. Preprint first posted 2023. arXiv:2308.08155

[21] Qian, C., et al. (2024). ChatDev: Communicative agents for software development. ACL, 15174–15186. 10.18653/v1/2024.acl-long.810

[22] Cemri, M., et al. (2025). Why do multi-agent LLM systems fail? Version 3. Preprint. arXiv:2503.13657v3

[23] Kim, Y., et al. (2026). Towards a science of scaling agent systems. Version 3; first posted 2025. Preprint. arXiv:2512.08296v3

[24] Anthropic (2025, 13 June). How we built our multi-agent research system. Engineering blog post, not peer reviewed. Web page

[25] Carlini, N. (2026, 5 February). Building a C compiler with a team of parallel Claudes. Anthropic engineering blog post, not peer reviewed. Web page

[26] Ohno, T. (1988). Toyota Production System: Beyond Large-Scale Production. Productivity Press. ISBN 9780915299140. Publisher record

[27] Automotive Industry Action Group (2008). Advanced Product Quality Planning and Control Plan (APQP). 2nd edition. AIAG. Library record

[28] Deming, W. E. (1986). Out of the Crisis. MIT Center for Advanced Engineering Study. Library record

[29] Harry, M., & Schroeder, R. (2000). Six Sigma: The Breakthrough Management Strategy Revolutionizing the World's Top Corporations. Currency Doubleday. Library record

[30] Shewhart, W. A. (1930). Economic quality control of manufactured product. Bell System Technical Journal, 9(2), 364–389. 10.1002/j.1538-7305.1930.tb00373.x

[31] Goldratt, E. M., & Cox, J. (1984). The Goal: A Process of Ongoing Improvement. North River Press. Library record

[32] Toyota Motor Corporation. Toyota Production System. Official company web page, accessed 29 October 2026. Web page

[33] Liker, J. K. (2004). The Toyota Way: 14 Management Principles from the World's Greatest Manufacturer. McGraw-Hill. Library record

[34] Couzin, I. D., Krause, J., Franks, N. R., & Levin, S. A. (2005). Effective leadership and decision-making in animal groups on the move. Nature, 433(7025), 513–516. 10.1038/nature03236

[35] Stapleton, D. (2026). Atlas Symphony. Author's project page, last updated 28 October 2026. Self-published record. Web page

[36] Stapleton, D. (2026). DHG Hive. Author's project page, last updated 28 October 2026. Self-published record. Web page

[37] AIEN (2026). Synapse swarm board. Snapshot 099a980fedab8a6797bbf0358b8c5502c8045142. Pinned mechanism documentation

[38] Malone, T. W., & Crowston, K. (1994). The interdisciplinary study of coordination. ACM Computing Surveys, 26(1), 87–119. 10.1145/174666.174668

[39] Conway, M. E. (1968). How do committees invent? Datamation, 14(4), 28–31. Author's reprint. Author reprint

[40] AIEN (2026). Crumb protocol RFC 0002. Snapshot c6512a4c6ae2e8f8babe23fc9f38880d20d25812. Pinned source

[41] Stapleton, D. (2026). AIEN operational record extract, 13–28 October 2026: coordination-board lines and incident handoffs with SHA-256 digests of the full private sources. Included in this article's source bundle as sources/operational_record_extract.md.

[42] Stapleton, M. Drake (2026). Reverse Engineering the CUDA Software Stack: An Independently Implemented GPU Machine-Code Toolchain for the NVIDIA GB10. Version 2. Preprint. 10.5281/zenodo.23677465

[43] Little, J. D. C. (1961). A proof for the queuing formula: L = λW. Operations Research, 9(3), 383–387. 10.1287/opre.9.3.383

[44] AIEN (2026). Synapse swarm board benchmark results. Snapshot 099a980fedab8a6797bbf0358b8c5502c8045142. Pinned results

[45] AIEN (2026). Finite swarm integration workstream plan. aien-architecture-mv issue 231. Repository issue, accessed 29 October 2026. Issue record

[46] AIEN (2026). Crumb RFC-0003 amendment 1: derive generated data on demand instead of committing it. aien-architecture-mv pull request 271, merged 18 October 2026 as 12ad0141b41ec67ef5aa51cb4e1815894c1083bb; migration pull requests aien-architecture-mv 272, aegis-runtime-mv 70, aien-protocols-mv 44, physics-mv 72, spark-rsi-mv 63 and synapse 117 merged 19 October 2026; aien-sovereign-core-mv 674 open on 29 October 2026. Pull request record

[47] Kapoor, S., Stroebl, B., Siegel, Z. S., Nadgir, N., & Narayanan, A. (2025). AI agents that matter. Transactions on Machine Learning Research. Published paper

[48] Jimenez, C. E., et al. (2024). SWE-bench: Can language models resolve real-world GitHub issues? ICLR. arXiv:2310.06770

[49] Zhou, S., et al. (2024). WebArena: A realistic web environment for building autonomous agents. ICLR. arXiv:2307.13854

[50] Hart, S. G., & Staveland, L. E. (1988). Development of NASA-TLX: Results of empirical and theoretical research. In Human Mental Workload, 139–183. 10.1016/S0166-4115(08)62386-9